Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3

Practical Number		CO/PO
3	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. **1. Distinct points (Point.java + Driver.java with main)** (a) Create class Point with private int x, y and a constructor. (b) Override toString() to return "(x, y)". (c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)). (d) In Driver.main: build a Point[] with some repeated coordinates. (e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not. (f) Print "Distinct: N". **Expected output:** *5 points with 2 repeats → "Distinct: 3".* **2. Duplicate card check (Card.java + Driver.java)** (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: <card>" the first time a repeat appears. **Expected output:** *a second "Ace of Spades" → "Duplicate found: Ace of Spades".* **3. (For Advanced Learners – home) Fractions (Fraction.java + Driver.java)** (a) Create class Fraction with private int num, den. (b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g. (c) Override toString() to return num + "/" + den. (d) Override equals()/hashCode() using the reduced num and den. (e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same. **Expected output:** *new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).* **Part B — Project Work (Hour 2): MiniBank** In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it. 1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance. 2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal. 3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method. 4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer. 5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. Why must equals() and hashCode() be overridden together? 2. What does the default Object.toString() return, and why override it? 3. What is the difference between a static nested class and an inner class? **Supplementary Problems –** • Make toString() also show whether the account is active.	**CO1** PO1, PO2, PO3

Practical Number		CO/PO
	• Add a method that returns a formatted, multi-line statement string for an account. **Key Skills to be addressed –** Overriding Object-class methods (toString, equals, hashCode, clone); the equals/hashCode contract; static nested classes; the instanceof operator. **Applications –** Readable, comparable objects are needed everywhere — logging, searching, and (later in this project) storing objects in collections. **Learning Outcome –** The student can make objects behave correctly for printing and comparison and can use nested classes. (CO1)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Two Account objects with the same accountNumber are created to test equality.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-03. Verify that two accounts with the same number are reported equal and that the toString() output is readable. (The @Override annotation is introduced later, in Practical 7.)	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct toString/equals/hashCode – 40%; nested class and clone – 25%; instanceof use – 15%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: two Account objects a1 and a2 created with the same accountNumber; one Customer object that has an Address. **Process:** equals() compares the accountNumber values; toString() builds the display string; instanceof checks the object's type. **Expected Output:** Printing a1 shows a readable line such as "AC0001 | Riya | 4000"; a1.equals(a2) prints true; the instanceof check prints true for an Account.	